

1.File Structure

```
project-folder/
├── assets/
│   ├── fonts/
│   ├── images/
│   └── sounds/
├── main.c
├── menu.c / menu.h
├── game.c / game.h
├── constants.h
├── Makefile
├── scores.txt
└── hints.txt
# External Resources
# .ttf files for UI and Game
# PNG textures for players and UI
# MP3/WAV files for music and SFX
# Entry point and Main Game Loop
# UI, Buttons, and Leaderboard logic
# World physics and Split-screen logic
# Global definitions (States, Screen Size)
# GCC Automation script
# Data file for leaderboard
# Text file for in-game 'H' key hints
```

2. Modular Overview

This project is a 2D game engine built in **C11** using the **SDL2** library suite. It uses a state-machine architecture to manage transitions between the menu, gameplay, and sub-menus.

Core Files:

- **main.c**: The entry point. Handles the main loop, 60 FPS capping, and state switching.
- **menu.c / .h**: Manages UI rendering, button logic, and the leaderboard system.
- **game.c / .h**: Manages the "World" logic, player movement, and camera viewports.
- **constants.h**: Defines global settings like screen resolution and GameState enums.

3. Function Reference Guide

UI & Menu Functions (menu.c)

Function	Description
create_button_text	Converts a string into an SDL texture and centers it inside a button's rectangle.
initMenu	Loads textures (background, logo, buttons), sounds, and initializes button positions.
displayMenu	The primary render loop for the main menu; handles button drawing and hover states.
displaySubMenu	Draws a generic "container" screen with a title (used for

Function	Description
	Story/Loading).
drawOptions	Renders the settings screen, including interactive checkboxes for sound and display.
handleMenuEvents	Processes mouse movement for hover sounds and clicks to change game states.
handleOptionsEvents	Manages checkbox logic for Fullscreen, Music Mute, and SFX volume.
freeMenu	Cleanup: Destroys all menu-related textures and frees music/SFX memory.
displayStory	Renders long blocks of text with automatic word-wrapping for the "Histoire" mode.
compareScores	A logic helper for qsort to rank players from highest to lowest score.
displayScores	Parses scores.txt, sorts entries, and renders the Leaderboard table.

Game & World Functions (game.c)

Function	Description
initGame	Initializes player coordinates, the world map texture, and the game timer.
handleGameInputs	Processes WASD (P1) and Arrow Keys (P2). Toggles split-screen with M.
updateGame	Calculates camera logic to ensure the "lens" follows the player within world bounds.
renderScene	Draws the background and player entities relative to a specific camera position.
drawGame	Viewport Management: Divides the screen if splitScreen is active and renders views.
drawTimer	Calculates elapsed time since the game started and renders a \$MM:SS\$ clock.
drawHint	Pulls a random line from hints.txt and displays it as a temporary overlay.
freeGame	Cleans up the game world texture and closing the game-specific fonts.

4. Build System Breakdown (Makefile)

The **Makefile** is your automation script. It prevents you from having to type complex compilation commands manually.

Key Variables

- **CC = gcc**: Sets the GNU Compiler Collection as the build tool.
- **CFLAGS**: Flags passed to the compiler. It includes -Wall (show all warnings) and sdl2-config --cflags (finds where SDL2 headers are installed on your system).
- **LDFLAGS**: Linker flags. These link the specific SDL2 "extension" libraries:
 - -ISDL2_image: For PNG/JPG support.
 - -ISDL2_ttf: For TrueType Font rendering.
 - -ISDL2_mixer: For MP3/WAV audio handling.

Available Commands

1. **make**: Compiles all .c files into object files (.o) and then links them into game_exec.
2. **make run**: Compiles the code (if changed) and immediately launches the game.
3. **make clean**: Removes all compiled files so you can perform a "hidden" or fresh build.

5. Execution Logic (main.c)

The program follows a strict lifecycle:

1. **Initialization**: SDL_Init and library setups.
2. **Asset Loading**: initMenu and initGame prepare all textures in RAM.
3. **The Loop**:
 - **Poll**: Check for SDL_PollEvent (Keyboard/Mouse).
 - **Update**: Change coordinates or game states.
 - **Render**: Clear screen -> Draw State -> SDL_RenderPresent.
 - **Delay**: Wait to maintain exactly **60 FPS**.
4. **Cleanup**: freeMenu and freeGame release hardware resources before the window closes.